

Chapter 8 - Objects

In the previous chapter we used tables as numbered lists, but we can also store values in a different way: With strings.

```
function love.load()
```

```
    --rect is short for rectangle
```

```
    rect = {}
```

```
    rect["width"] = 100
```

```
end
```

"width" in this case is what we call a *key* or a *property*. So the table rectangle now has the property "width" with a value of 100. We don't need to use strings every time we want to create property. A dot (.) is the shorthand for table_name["property_name"].

```
function love.load()
```

```
    rect = {}
```

```
    -- These two are the same
```

```
    rect["width"] = 100
```

```
    rect.width = 100
```

```
end
```

Let's add some more properties.

```
function love.load()
```

```
    rect = {}
```

```
    rect.x = 100
```

```
    rect.y = 100
```

```
    rect.width = 70
```

```
    rect.height = 90
```

```
end
```

Now that we have our properties we can start drawing the rectangle.

```
function love.draw()
```

```
love.graphics.rectangle("line", rect.x, rect.y, rect.width, rect.height)
```

```
end
```

And let's make it move!

```
function love.load()
```

```
    rect = {}
```

```
    rect.x = 100
```

```
    rect.y = 100
```

```
    rect.width = 70
```

```
    rect.height = 90
```

```
    --Add a speed property
```

```
    rect.speed = 100
```

```
end
```

```
function love.update(dt)
```

```
    -- Increase the value of x. Don't forget to use delta time.
```

```
    rect.x = rect.x + rect.speed * dt
```

```
end
```

Now we have a moving rectangle again, but to show the power of tables I want to create multiple moving rectangles. For this we're going to use a table as a list. We'll make a list of rectangles. Move the code inside the love.load to a new function, and create a new table in love.load.

```
function love.load()
```

```
    -- Remember: camelCasing!
```

```
    listOfRectangles = {}
```

```
end
```

```
function createRect()
```

```
    rect = {}
```

```
    rect.x = 100
```

```
    rect.y = 100
```

```
    rect.width = 70
```

```
    rect.height = 90
```

```
    rect.speed = 100
```

```
    -- Put the new rectangle in the list
```

```
    table.insert(listOfRectangles, rect)
```

```
end
```

So now every time we call createRect, a new rectangle object will be added to our list. That's right, a table filled with tables. Let's make it so that whenever we press space, we call createRect. We can do this with the callback love.keypressed.

```
function love.keypressed(key)
```

```
    -- Remember, 2 equal signs (==) for comparing!
```

```
    if key == "space" then
```

```
        createRect()
```

```
    end
```

```
end
```

Whenever we press a key, LÖVE will call love.keypressed, and pass the pressed key as argument. If that key is "space", it will call createRect.

Last thing to do is to change our update and draw function. We have to iterate through our list of rectangles.

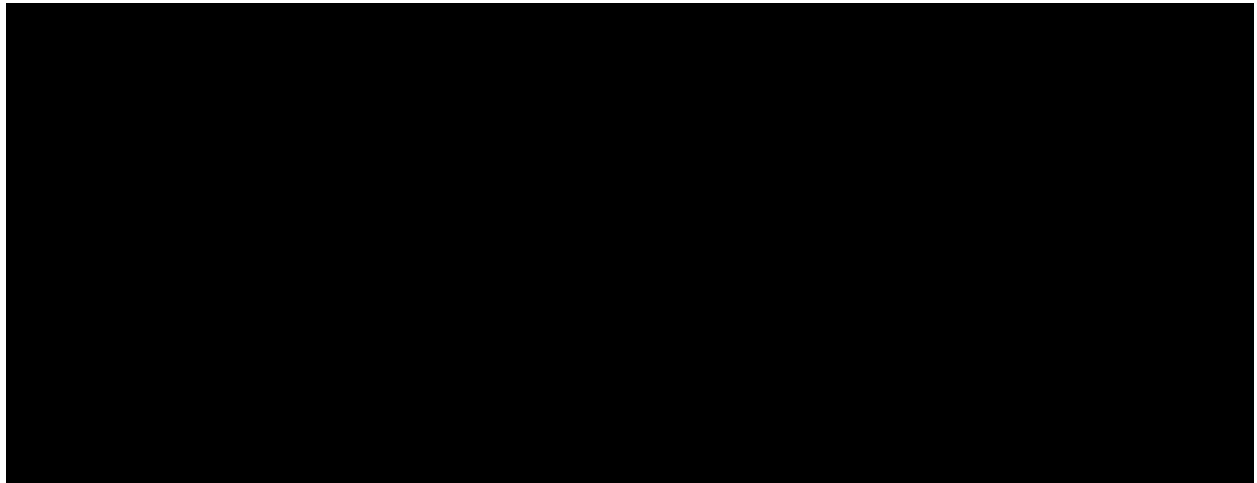
```
function love.update(dt)
```

```
    for i,v in ipairs(listOfRectangles) do
```

```
        v.x = v.x + v.speed * dt
```

```
end  
end  
  
function love.draw(dt)  
  for i,v in ipairs(listOfRectangles) do  
    love.graphics.rectangle("line", v.x, v.y, v.width, v.height)  
  end  
end
```

And now when you run the game, a moving rectangle should appear every time you press space.



One more time?

That was a lot of code in a rather short tutorial. I can imagine it can be quite confusing so let's go through the whole code one more time:

In `love.load` we created a table called `listOfRectangles`.

When we press space, LÖVE calls `love.keypressed`, and inside that function we check if the pressed key is "space". If so, we call the function `createRect`.

In `createRect` we create a new table. We give this table properties, like `x` and `y`, and we store this new table inside our list `listOfRectangles`.

In love.update and love.draw we iterate through this list of rectangles, to update and draw each rectangle.

Functions

An object can also have functions. You create a function for an object like this:

```
tableName.functionName = function ()
```

```
end
```

-- Or the more common way

```
function tableName.functionName()
```

```
end
```

Summary

We can store values in tables not only with numbers but also with strings. We call these type of tables *objects*. Having objects saves us from creating a lot of variables.